

A printable quick reference for everyday SQL — reading, summarising and joining data. Examples use standard / SQLite-style syntax; a few dialect differences are flagged at the end.

THE BASICS	
SELECT a, b	Choose the columns to return.
SELECT *	Return every column.
FROM t	The table to read from.
AS name	Rename a column or table (alias).
;	Ends a statement.
-- text	Single-line comment (/* ... */ block).

CLAUSE ORDER	
Written:	SELECT > FROM > WHERE > GROUP BY > HAVING > ORDER BY > LIMIT
Runs as:	FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT
Why it matters	SELECT runs late — so a column alias can't be used in WHERE.

FILTER ROWS — WHERE	
WHERE region = 'EMEA'	Keep matching rows. Text in single quotes.
= <> > < >= <=	Comparison operators (<> = not equal).
AND / OR / NOT (...)	Combine or negate conditions. Group conditions to control precedence.

MATCH & RANGES	
LIKE 'SD-WAN%'	Pattern: % = any text, _ = one char.
IN ('A', 'B', 'C')	Equals any value in a list.
BETWEEN 10 AND 20	Inclusive range (numbers or dates).
IS NULL / IS NOT NULL	Test for missing values.
NOT IN / NOT LIKE	Negate a match.

SORT, LIMIT, DEDUPE	
ORDER BY revenue DESC	Sort high-to-low (ASC = default).
ORDER BY 2	Sort by the 2nd selected column.
LIMIT 10	Return only the first 10 rows.
LIMIT 10 OFFSET 20	Skip 20, then take 10 (paging).
SELECT DISTINCT col	Return each value once.

AGGREGATE FUNCTIONS	
COUNT(*)	Number of rows.
COUNT(col)	Non-NULL values in a column.
COUNT(DISTINCT col)	Number of distinct values.
SUM / AVG(col)	Total / mean.
MIN / MAX(col)	Smallest / largest.
ROUND(x, 1)	Round to n decimals.

GROUP BY & HAVING	
GROUP BY region	One result row per group.
SUM(rev) AS total	Aggregate within each group; alias it.
HAVING COUNT(*) >= 10	Filter groups after aggregating.
WHERE vs HAVING	WHERE filters rows first; HAVING filters groups.

JOINS	
FROM a JOIN b ON a.id=b.id	INNER join — only matching rows.
LEFT JOIN b ON ...	All left rows + matches (NULLs if none).
RIGHT / FULL JOIN	All right / all rows from both sides.
CROSS JOIN	Every combination (no ON).
FROM orders o JOIN ...	Alias tables to keep refs short (o.col).
USING (region)	Shorthand when the key name matches.

SET OPERATORS	
UNION	Stack two results, remove duplicates.
UNION ALL	Stack, keep duplicates (faster).
INTERSECT	Rows present in both.
EXCEPT	Rows in the first but not the second.

SUBQUERIES	
WHERE x > (SELECT AVG(x) ...)	Scalar — a single value.
WHERE id IN (SELECT id ...)	List — match against many values.
WHERE EXISTS (SELECT 1 ...)	True if the subquery returns any row.
FROM (SELECT ...) AS s	Derived table — query a query.

CTE — WITH	
WITH t AS (SELECT ...)	Name a temporary result set, build it once,
SELECT * FROM t	then use it like a table — clearer than nesting.
WITH a AS(...), b AS(...)	Chain several CTEs with commas.

WINDOW FUNCTIONS	
ROW_NUMBER() OVER (ORDER BY x)	Sequential number per row.
RANK() / DENSE_RANK()	Ranking (with / without gaps).
SUM(x) OVER (PARTITION BY g)	Group total shown on every row.
SUM(x) OVER (ORDER BY d)	Running total.
LAG / LEAD(x) OVER (ORDER BY d)	Previous / next row's value.

CASE — IF/THEN	
CASE WHEN x>=30000	Start a conditional expression.
THEN 'big'	Result when the test is true.
ELSE 'small' END	Fallback; END closes it.
Use it in SELECT	...or inside SUM() to pivot, or in ORDER BY.

TEXT FUNCTIONS	
UPPER(s) / LOWER(s)	Change case.
LENGTH(s)	Number of characters.
SUBSTR(s, start, len)	Slice out part of a string.
TRIM(s)	Strip leading/trailing spaces.
REPLACE(s, old, new)	Swap text by content.
a b	Concatenate (CONCAT() in some dialects).

NUMBERS & MATH	
ROUND(x, n) / ABS(x)	Round / absolute value.
CEIL(x) / FLOOR(x)	Round up / down to integer.
a % b	Remainder (MOD(a,b) in some dialects).
100.0 * a / b	Force decimals — integer / integer truncates.

DATES (VARY BY DIALECT)

CURRENT_DATE / NOW()	Today / current timestamp.
SQLite	DATE('now'), strftime('%Y', d)
PostgreSQL	DATE_PART('year', d), d - INTERVAL '7 day'
SQL Server	GETDATE(), DATEPART(year, d)
Tip	Store dates as DATE/ISO text, not free text.

NULL HANDLING

IS NULL / IS NOT NULL	The only safe NULL test.
x = NULL	Never matches — NULL is 'unknown'.
COALESCE(a, b, 0)	First value that isn't NULL.
NULLIF(a, b)	NULL when a equals b (avoid /0).

CREATE & CONSTRAIN (DDL)

CREATE TABLE t (	Define a new table:
id INTEGER PRIMARY KEY,	types: INTEGER, REAL, TEXT, DATE, BOOLEAN
region TEXT NOT NULL,	NOT NULL · UNIQUE · DEFAULT v
cust_id INT REFERENCES c(id))	FOREIGN KEY links tables.
DROP / ALTER TABLE	Remove a table / add a column.

MODIFY DATA (DML)

INSERT INTO t (a,b) VALUES (1, 'x')	Add a row.
INSERT INTO t SELECT ...	Insert query results.
UPDATE t SET a=1 WHERE id=5	Change rows — always add WHERE!
DELETE FROM t WHERE id=5	Remove rows — WHERE or you clear all.

COMMON PATTERNS

Top-N per group	ROW_NUMBER() OVER(PARTITION BY g ORDER BY x DESC), keep n<=3
% of total	100.0 * x / SUM(x) OVER ()
Running total	SUM(x) OVER (ORDER BY d)
Pivot a column	SUM(CASE WHEN region='EMEA' THEN rev END)
De-duplicate	GROUP BY key —or— SELECT DISTINCT

GOTCHAS

'text' vs "ident"	Single quotes = text; double quotes = column/table names.
= NULL	Use IS NULL — equality never matches NULL.
Integer division	3/2 = 1; multiply by 1.0 for decimals.
COUNT(col)	Skips NULLs; COUNT(*) counts all rows.
Aggregates in WHERE	Not allowed — use HAVING.

DIALECT QUICK NOTES

Limit rows	LIMIT n (SQLite/PG/MySQL) · TOP n (SQL Server) · FETCH FIRST n ROWS
Concatenate	a b (standard/SQLite/PG) · CONCAT(a,b) (MySQL/SQL Server)
Quote a name	"col" standard · [col] SQL Server · `col` MySQL
Case sensitivity	LIKE is case-insensitive in some engines, not others.

TRANSACTIONS

BEGIN; ... COMMIT;	Group changes and save them together.
ROLLBACK;	Undo everything since BEGIN.
Use for multi-step writes	All-or-nothing — no half-finished updates.

VIEWS & INDEXES

CREATE VIEW v AS SELECT ...	Save a query as a reusable virtual table.
SELECT * FROM v	Query the view like any table.
CREATE INDEX ix ON t(col)	Speed up filters and joins on a column.
Trade-off	Faster reads, slightly slower writes.

PERFORMANCE TIPS

Filter early	WHERE before joins/aggregates cuts the work.
Avoid SELECT *	Pull only the columns you need.
Index join & filter keys	Big speed-up on large tables.
EXPLAIN your query	See how the engine plans to run it.
Keep functions off filters	WHERE YEAR(d)=2025 can skip an index.

WINDOW FRAMES

... OVER (ORDER BY d	Add a frame to a window function:
ROWS BETWEEN 2 PRECEDING	a 3-row moving window
AND CURRENT ROW)	e.g. a 3-period moving average.
UNBOUNDED PRECEDING	From the first row (running total).

STRING AGGREGATION

GROUP_CONCAT(col, ', ')	SQLite / MySQL: join a group into one string.
STRING_AGG(col, ', ')	PostgreSQL / SQL Server equivalent.
With GROUP BY	One combined value per group.

CONDITIONAL AGGREGATION

SUM(CASE WHEN c THEN 1 END)	Count rows meeting a condition.
AVG(CASE WHEN c THEN x END)	Aggregate just a subset, inline.
One CASE per column	Turns rows into a pivot table.

READING A QUERY

Read inside-out	Start at FROM and the innermost subquery.
Build up gradually	Add one clause, run, check, repeat.
SELECT * first	See raw rows before you aggregate.
Comment to isolate	-- out lines to find a problem.

TABLES & RELATIONSHIPS

Primary key	Uniquely identifies each row.
One-to-many	Foreign key lives on the 'many' side.
Many-to-many	Add a junction table holding both keys.
See Foundations	for ERDs & schema design.